Krum, une bibliothèque pour l'agrégation robuste

Apprentissage distribué robuste

Arthur Danjou

M2 ISF, Université Paris-Dauphine – PSL
Stage au CMAP, École polytechnique, encadré par El-Mahdi El-Mhamdi

Soutenance du 2 novembre 2026

Le CMAP et le groupe Robust Learning

Centre de Mathématiques Appliquées (CMAP)
Laboratoire de l'École polytechnique.

Le groupe **Robust Learning** étudie l'apprentissage robuste.

- **El-Mahdi El-Mhamdi** encadre les travaux scientifiques.
- **Peva Blanchard** fixe les priorités du produit.
- **Sébastien Rouault** assure la direction technique.
- **Mohamad-Ammar Said et moi** contribuons au développement.

Les contributions sont relues avant leur intégration, et les vérifications se font automatiquement.

Le sujet du stage

J'ai participé au développement de **Krum**, une bibliothèque de composants logiciels pour mener des expériences en **apprentissage distribué robuste**.

Le problème étudié

Plusieurs machines entraînent un même modèle. Certains participants sont malhonnêtes et faussent les calculs transmis.

Plan

1. Le problème et la mission.
2. La construction de Krum.
3. Les validations.
4. Le bilan du stage.

Apprentissage supervisé et entraînement distribué

En **apprentissage supervisé**, chaque exemple comprend une entrée et la réponse attendue.

1. Le modèle **prédit une réponse** à partir de l'entrée.
2. La **fonction de perte** (*loss*) mesure l'écart avec la réponse attendue.
3. On **ajuste les paramètres du modèle** pour minimiser la perte sur les exemples d'entraînement.

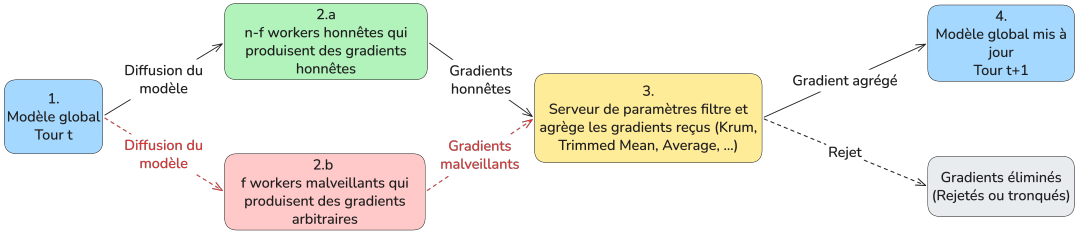
Pourquoi distribuer l'entraînement ?
Pour garder les données sur place ou répartir un calcul trop lourd pour une seule machine.

Claviers de téléphones
Plusieurs appareils apprennent à prédire le mot suivant sans centraliser les textes.

Hôpitaux
Plusieurs établissements entraînent un modèle de diagnostic en gardant les dossiers sur place.

Un tour d'apprentissage centralisé

Un *worker* est un participant qui effectue un calcul local.



La **descente de gradient stochastique** (SGD) estime le gradient sur un **mini-lot** d'exemples locaux, puis met à jour le modèle dans la direction opposée à ce gradient.

En **décentralisé**, les participants mettent à jour leurs modèles par des échanges entre voisins, sans serveur central.

Des participants malhonnêtes

Trois surfaces d'attaque

Les données
L'attaquant altère les exemples.

Le serveur
L'attaquant compromet le point central.

Les gradients envoyés
Des workers malhonnêtes forgent leurs contributions.

Nous étudions les **gradients forgés**. Dans le **scénario byzantin**, parmi les n workers, au plus f sont **malhonnêtes**. Ils peuvent envoyer des vecteurs arbitraires.

- **Collusion** : les attaquants coordonnent leurs vecteurs.
- **Omniscience** : ils connaissent l'état du système, dont les gradients honnêtes.

Le serveur doit agréger ces vecteurs sans savoir lesquels viennent de participants malhonnêtes.

Pourquoi remplacer la moyenne ?

Un attaquant peut déplacer la moyenne aussi loin qu'il le souhaite en envoyant un vecteur de grande amplitude dans la direction de son choix.

Blanchard et al., NeurIPS 2017, lemme 1.

Agrégation robuste

La règle doit limiter l'influence des attaquants tout en fournissant une direction utile à chaque tour d'optimisation.

Du copier-coller à des composants réutilisables

Des chercheurs reprenaient le code de Sébastien par copier-coller, sans toujours le comprendre.

Mon audit initial et mes reproductions centralisées de zéro m'ont confronté à ces difficultés.

La mission initiale du stage

Réunir les outils nécessaires pour reproduire et comparer ces expériences.

Krum : une bibliothèque en trois couches

Krum est une bibliothèque Python fondée sur **PyTorch**, un outil de calcul et d'entraînement des modèles.

Primitives Briques de calcul.

Simulations Exécution des protocoles.

Orchestration Gestion des campagnes d'expériences.

Mes contributions dans ce projet collectif

J'ai développé des agrégateurs et travaillé sur le composant `Model`, les architectures de modèles et les simulations centralisés. Le composant `Model` donne accès aux paramètres sans les recopier. J'ai aussi entièrement rédigé la documentation et les tutoriels.

Sébastien Rouault et Peva Blanchard ont développé l'orchestrateur.

1. Primitives

1.1 Agrégateurs

1.2 Attaques

1.3 Partitionnement des données

1.4 Extension des primitives

1.5 Modèles pour l'agrégation

1.1 Les familles d'agrégateurs

J'ai pris en charge leur implémentation et leurs tests.

Distances	Sélection de gradients proches.
Coordonnées	Filtrage de chaque coordonnée séparément.
Hybrides	Combinaison de sélection et de filtrage.
Géométrie	Recherche d'un centre des vecteurs.

La moyenne comme référence : `Average`

Pour les gradients reçus $\boldsymbol{g}_1, \ldots, \boldsymbol{g}_n$,

$$\text{Average} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{g}_i.$$

La moyenne inclut toutes les contributions, même arbitraires.

1.1 `MultiKrum` : sélectionner puis moyenner

Au plus f des n participants sont **malhonnêtes**.

$\mathcal{N}_i$ contient les $n - f - 2$ plus proches voisins de $\boldsymbol{g}_i$ parmi les **autres** gradients.

$$s_i = \sum_{j \in \mathcal{N}_i} \|\boldsymbol{g}_i - \boldsymbol{g}_j\|^2.$$

On calcule les scores une seule fois. I_m contient les indices des m gradients ayant les scores les plus faibles.

$$\text{MultiKrum}(\boldsymbol{g}_1, \ldots, \boldsymbol{g}_n) = \frac{1}{m} \sum_{i \in I_m} \boldsymbol{g}_i.$$

Pour $m = 1$, on retrouve `Krum`. Pour $m = n$, on retrouve **Average**.

1.2 Deux attaques pour éprouver les agrégateurs

Rappel : scénario d'omniscience.
Les attaquants ont accès aux gradients honnêtes.

On note $\bar{\boldsymbol{g}}$ la moyenne des gradients honnêtes.

`SignFlip`

L'attaque inverse et amplifie la moyenne des gradients honnêtes.

$$\boldsymbol{b} = -\lambda \bar{\boldsymbol{g}}, \qquad \lambda > 0.$$

Tous les participants malhonnêtes envoient ce même vecteur.

`SmallPerturbation`

L'attaque décale une seule coordonnée de la moyenne honnête.

$$\boldsymbol{b} = \bar{\boldsymbol{g}} + \gamma \boldsymbol{e}_k.$$

$\boldsymbol{e}_k$ est le vecteur de base de la coordonnée k. Le vecteur cherche à rester sélectionnable.

1.3 Les données locales peuvent aussi différer

Données IID

Observations indépendantes et de même loi.

Répartition aléatoire en sous-jeux comparables.

Données non IID

Les lois locales peuvent différer.

Exemple : les hôpitaux accueillent des populations différentes.

Partitionneurs

Distribuer les données entre participants et régler leur hétérogénéité pour comparer les agrégateurs.

Même sans attaque, les gradients honnêtes peuvent différer.

1.4 Une seule méthode pour ajouter une règle

Une méthode est une fonction associée à un composant logiciel.

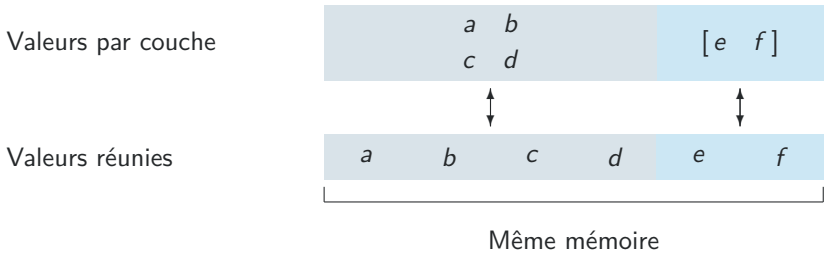
Primitive	**Méthode à implémenter**
Agrégateur	`aggregate`
Attaque	`generate`
Partitionneur	`partition`

L'interface commune permet de réutiliser la primitive dans les simulations sans réécrire leur boucle.

Au total, la bibliothèque comprend onze règles, cinq attaques, quatre partitionneurs et six architectures de modèles.

1.5 Préparer les modèles pour l'agrégation

PyTorch stocke les paramètres et les gradients par couche. Pour les agréger, le composant `Model` les présente sous forme d'une suite de valeurs.



La vue réunie partage la mémoire avec les valeurs par couche, ce qui évite de les recopier pour l'agrégateur. Le calcul des gradients reste nécessaire.

2. Simulations

La simulation prend en charge la boucle d'entraînement

En reproduisant des expériences à partir de zéro, j'ai dû réécrire la même boucle d'entraînement.

L'utilisateur choisit la répartition des données entre participants.

La simulation automatise le protocole

1. Former et tirer les petits lots de données.
2. Calculer les gradients locaux.
3. Appliquer l'attaque et l'agrégation.
4. Mettre à jour le modèle.

Fidélité des simulations

L'objectif

La simulation permet de faire varier les paramètres de l'expérience tout en gardant la même boucle d'entraînement sous PyTorch.

Fidélité au protocole

Une simulation fidèle conserve les mécanismes du protocole nécessaires à l'étude du phénomène visé et documente les adaptations apportées.

3. Orchestration

L'orchestrateur gère les campagnes

L'orchestrateur lance les combinaisons de paramètres et enregistre les résultats.

À la relance, il ne reprend que les expériences nouvelles, échouées ou périmées.

Développé par Sébastien Rouault et Peva Blanchard.

4. Qualité logicielle

Qualité et distribution de la bibliothèque

- Plusieurs centaines de tests automatisés vérifient le comportement des composants.
- J'ai entièrement rédigé la documentation et les tutoriels pour expliquer comment utiliser et étendre la bibliothèque.

Licence MIT `pip install krum`

5. Expériences

2017 : `MultiKrum` face à la moyenne

La sélection maintient-elle l'apprentissage sous attaque ?

Données et modèle

Spambase : courriels, 57 variables, deux catégories.
Deux couches cachées de 20 neurones.

Scénario byzantin

20 workers, dont 6 malhonnêtes.
`SignFlip` inverse la moyenne des gradients honnêtes et l'amplifie par 10.

Données indépendantes et de même loi (IID).
300 tours. Pas 0,01. Graine 42.

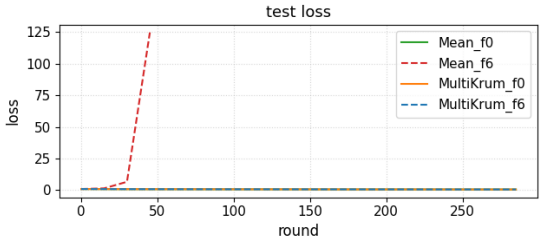
Un réglage hors garantie

Sous attaque : $m = n - f = 14$.
Sans attaque : $m = n$, donc `MultiKrum` coïncide avec la moyenne.

Légendes : `Mean` désigne `Average` ; `f0` et `f6` indiquent 0 et 6 malhonnêtes.

Blanchard et al., NeurIPS 2017. Protocole adapté, observation hors garantie.

2017 : la perte de la moyenne diverge

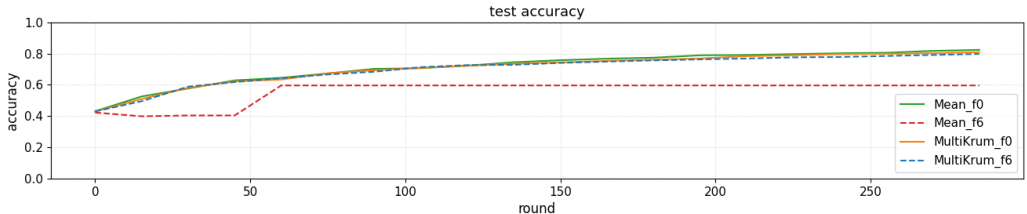


Moyenne attaquée

À partir du tour 40, les poids deviennent indéfinis (NaN).

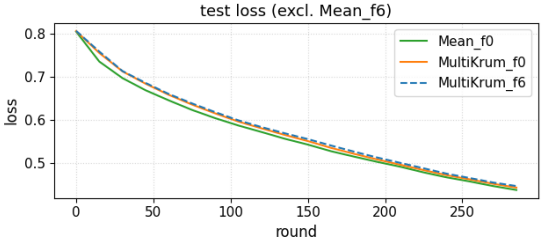
Les 60 % reflètent la classe majoritaire.

Le modèle n'a plus de performance valide.



Blanchard et al., NeurIPS 2017. Perte test en haut, exactitude test en bas.

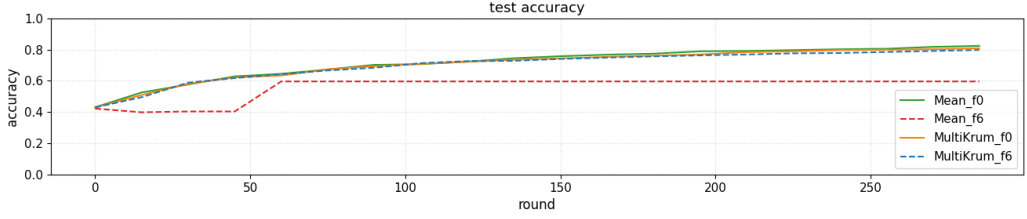
2017 : `MultiKrum` continue à apprendre



Zoom de 0,4 **à** 0,8**.**
La moyenne attaquée est exclue.

Exactitude : environ **77 % sous attaque**, contre **82 % sans attaque**.

$m = 14$, hors garantie.



Blanchard et al., NeurIPS 2017. Perte test en haut, exactitude test en bas.

2018 : une perturbation peut passer la sélection

Un gradient proche des honnêtes peut-il dégrader `Krum` ?

Données et modèle

MNIST : images de chiffres manuscrits.

Une couche cachée de 100 neurones.

Scénario byzantin

39 workers, dont 9 malhonnêtes.

`SmallPerturbation` décale une coordonnée, $\gamma = 4$ fixé.

Le vecteur reste sélectionné à chaque tour.

Données IID. 300 tours. Graine 42.

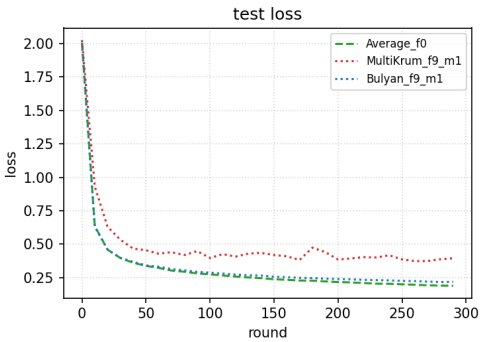
`MultiKrum` avec $m = 1$ est `Krum` ; `Bulyan` subit la même attaque.

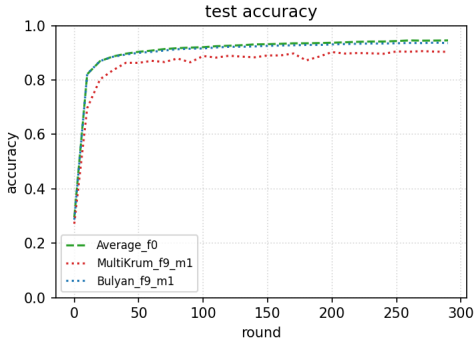
`Bulyan` ajoute un second filtre

1. Sélection de vecteurs selon leurs distances.
2. Pour chaque coordonnée, moyenne des valeurs les plus proches de la médiane.

El-Mhamdi et al., ICML 2018. Protocole adapté.

2018 : Krum apprend, puis plafonne





Krum attaqué	Bulyan attaqué	Moyenne sans attaque
$\approx$ **90** %	$\approx$ **94** %	$\approx$ **95** %

La perturbation passe la sélection de Krum à chaque tour et dégrade peu à peu le résultat.

Exactitude finale. El-Mhamdi et al., ICML 2018.

Ce que ces expériences permettent de conclure

Les expériences retrouvent les phénomènes visés, avec des adaptations documentées.

À code, graines, matériel et environnement déterministe fixés, deux exécutions donnent les mêmes résultats bit à bit.

Moins de trois minutes par expérience sur CPU, le processeur de la machine.

L'équipe a aussi implémenté `MoNNA`, un protocole décentralisé (Farhadkhani et al., ICML 2023).

6. Positionnement et limites

Face à cinq bibliothèques comparables

Pour la comparaison, j'ai retenu ByzFL, FedLab, Blades, FL-Byzantine-Library et ByzPy, dont les composants d'agrégation byzantine sont ouverts et comparables.

Critère	**Krum**	**Dans la comparaison**
Catalogue	11 règles et 5 attaques.	Certaines bibliothèques proposent plus de composants.
Zéro-copie	`Model` partage la mémoire.	L'accès copie les valeurs ou n'a pas d'équivalent.
Protocoles	Trois protocoles documentés.	Cadres génériques ou suites.
Relance	Recalcul des résultats périmés.	ByzFL saute les exécutions terminées.

J'ai analysé ces composants.

Comptes d'août 2026, audités en septembre 2026.

Les limites actuelles

- Le catalogue reste ciblé. Certaines bibliothèques proposent plus de composants.
- Les pré-agrégateurs ne sont pas inclus. Appliqués avant l'agrégation, ils peuvent borner les normes ou réduire le bruit.
- Les expériences s'exécutent séquentiellement sur une seule machine. Les échanges entre participants sont simulés. On évalue donc les protocoles, pas un déploiement distribué réel.

7. Bilan et perspectives

Mes contributions au projet

Mon travail a porté sur les agrégateurs, `Model`, les architectures de modèles et les simulations centralisés. J'ai écrit les tests associés automatisés.

J'ai aussi entièrement rédigé la documentation et les tutoriels. J'ai assuré une partie de la rédaction du papier compagnon, puis l'ai corrigé avec El-Mahdi El-Mhamdi.

Le papier est en cours de rédaction. Il vise la rubrique logiciels libres du *Journal of Machine Learning Research* (JMLR).

Le stage m'a appris à justifier mes décisions d'architectures et à faire relire mon code. J'ai commencé à apprendre à lire des articles de recherche. J'ai aussi découvert la rédaction d'un article scientifique.

Les acquis du stage et la suite

Krum réunit des composants réutilisables pour reproduire et comparer des expériences d'agrégation robuste.

Les acquis ISF en pratique

J'ai relié statistiques et optimisation au code testé, puis à l'analyse des résultats.
Le travail en équipe m'a appris à expliquer et documenter mes choix.

La thèse dans la chaire ATLAS

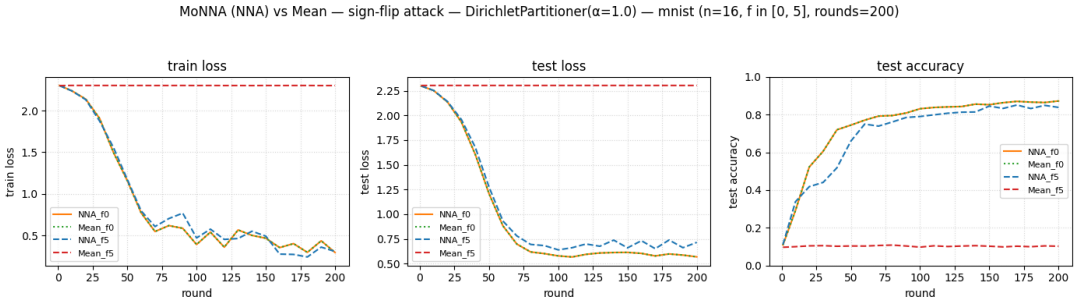
J'étudierai les garanties des agrégateurs pour la sécurité de l'apprentissage.
Krum servira à mener des campagnes expérimentales comparables.

Merci à l'équipe et merci de votre attention.

MoNNA décentralisé : étude de cas ICML 2023

L'équipe a réalisé cette simulation et analysé ses résultats. Mon travail portait sur le cas centralisé.

MNIST réduit : $n = 16$, $f = 5$, 200 tours, `SignFlip` d'amplitude 1.



La règle `NNA` moyenne les modèles les plus proches du modèle local.

Exactitude finale sous attaque : `NNA` $\approx 84\,\%$, `Average` $\approx 10\,\%$. Sans attaque : $\approx 87\,\%$.

Farhadkhani et al., ICML 2023.

Une expérience centralisée

```
sim = KrumSimulation(
    model_cls=Krum2017MLPMnist,
    train_datasets=worker_datasets, test_set=test_set,
    aggregator=aggregator, attack=attack,
    n=n, f=f, rounds=100, batch_size=32, lr=0.1, seed=seed,
)
sim.setup()

loss = Metric("loss", dtype=float)
accuracy = Metric("accuracy", dtype=float)
for step in range(100):
    sim.step()
    if step % 10 == 0:
        test_loss, test_accuracy = sim.evaluate()
        loss.push(step, test_loss, skip_if_exists=True)
        accuracy.push(step, test_accuracy, skip_if_exists=True)
```

Faire varier les configurations d'une campagne

```
with Orchestrator("krum_byzantine_study") as orch:
    for n, f in [(13, 2), (25, 5)]:
        for aggregator in [Average, Krum, Bulyan]:
            for attack in [ALIEAttack, SignFlipAttack, None]:
                orch.run(
                    experiment, n=n, f=f,
                    aggregator=aggregator, attack=attack, seed=42,
                )

loss_df = orch.get("loss").to_pandas()
acc_df = orch.get("accuracy").to_pandas()
```

`None` signifie qu'aucune attaque n'est appliquée. Les mesures sont converties en tableaux avec la bibliothèque pandas. L'orchestrateur a été développé par Sébastien Rouault et Peva Blanchard.

Extrait du listing de l'annexe 08.

Notations des agrégateurs : moyenne et coordonnées

Soient $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^d$ les gradients reçus. Pour une coordonnée k, $x_{(1),k} \leq \cdots \leq x_{(n),k}$ désigne les valeurs triées.

$$\texttt{Average}: \quad A = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}_i.$$

$$\texttt{Median}: \quad M_k = \mathrm{med}_{i=1,\ldots,n}\, x_{i,k}.$$

$$\texttt{TrimmedMean}: \quad T_k = \frac{1}{n-2f} \sum_{r=f+1}^{n-f} x_{(r),k}.$$

`TrimmedMean` retire donc les f valeurs extrêmes de chaque côté, séparément pour chaque coordonnée.

Notations des agrégateurs : distances et filtrage

Pour chaque i, $\mathcal{N}_i$ contient les $n - f - 2$ gradients les plus proches de $\boldsymbol{x}_i$ parmi les autres gradients.

$$s_i = \sum_{j \in \mathcal{N}_i} \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2, \qquad i^* = \arg\min_i s_i.$$

$$\texttt{Krum}: \quad \boldsymbol{x}_{i^*}.$$

$$\texttt{MultiKrum}: \quad \frac{1}{m} \sum_{i \in I_m} \boldsymbol{x}_i, \quad I_m = \text{indices des } m \text{ plus petits scores } s_i.$$

Les scores sont calculés une seule fois dans la bibliothèque.

`Bulyan` sélectionne itérativement $\theta = n - 2f - 2$ candidats avec `MultiKrum`. Pour chaque coordonnée k, il moyenne les $\theta - 2f$ valeurs sélectionnées les plus proches de leur médiane.

$$B_k = \frac{1}{\theta - 2f} \sum_{r \in J_k} x_{(r),k}, \qquad |J_k| = \theta - 2f.$$

Notations des agrégateurs : géométrie et recherche

$$\texttt{GeoMed}: \quad \arg\min_{y\in\mathbb{R}^d} \sum_{i=1}^{n} \|y - x_i\|.$$

$$\texttt{Medoid}: \quad \arg\min_{x\in\{x_1,\dots,x_n\}} \sum_{i=1}^{n} \|x - x_i\|.$$

$$\texttt{Brute}: \quad \frac{1}{|S^*|}\sum_{i\in S^*} x_i, \quad S^* = \arg\min_{\substack{S\subseteq\{1,\dots,n\}\\ |S|=n-f}} \max_{i,j\in S} \|x_i - x_j\|.$$

`GeoMed` minimise la somme des distances sans imposer que la sortie soit un gradient reçu. `Medoid` impose cette contrainte. `Brute` choisit le sous-ensemble de diamètre minimal.

Notations des agrégateurs : pivot et voisinage

`AKSEL` prend pour pivot $\boldsymbol{p}$ la médiane coordonnée par coordonnée, puis moyenne les $n - f$ gradients les plus proches de ce pivot.

$$p_k = \mathrm{med}_{i=1,\dots,n}\, x_{i,k}, \qquad \mathrm{AKSEL}(\boldsymbol{x}_{1:n}) = \frac{1}{n-f} \sum_{i \in I_{n-f}} \boldsymbol{x}_i, \quad I_{n-f} = \text{indices les plus proches de } \boldsymbol{p}.$$

Pour `NNA`, $\boldsymbol{z}^{(0)}$ est le pivot et q est le nombre de vecteurs à agréger, paramètre `num_closest`.
L'application τ ordonne les autres vecteurs reçus par distance croissante au pivot, de sorte que la sortie inclut le pivot et ses $q - 1$ plus proches voisins :

$$\mathrm{NNA} = \frac{1}{q} \sum_{r=0}^{q-1} \boldsymbol{z}^{(\tau(r))}, \qquad \tau(0) = 0.$$

Dans `MoNNA`, le modèle local sert de pivot et $q = n - 2f$.

Notations des attaques : gradients honnêtes et statistiques

On note $\mathcal{H}$ l'ensemble des gradients honnêtes, $|\mathcal{H}| = n - f$, et $\bar{\boldsymbol{g}} = \frac{1}{n-f} \sum_{i \in \mathcal{H}} \boldsymbol{g}_i$. Pour chaque coordonnée k, μ_k et σ_k sont la moyenne et l'écart-type honnêtes.

$$
\begin{aligned}
\texttt{SignFlip} &: \quad \boldsymbol{b} = -c\,\bar{\boldsymbol{g}}. \\
\texttt{ALIE} &: \quad b_k = \mu_k \pm z\sigma_k, \quad \text{avec le signe choisi pour l'attaque.} \\
\texttt{Gaussian} &: \quad \boldsymbol{b} \sim \mathcal{N}(\mu_G \mathbf{1}, \sigma_G^2 I_d), \quad \text{indépendamment des gradients honnêtes.}
\end{aligned}
$$

Les f participants malveillants envoient chacun le vecteur ainsi défini. Pour `ALIE`, z règle l'écart aux statistiques honnêtes. Pour `Gaussian`, μ_G et σ_G sont les paramètres du bruit.

Notations des attaques : négation et perturbation

`FullGradientNegation` suppose connu le gradient calculé sur l'ensemble des données :

$$\boldsymbol{b} = -\kappa \boldsymbol{g}_{\text{full}}.$$

`SmallPerturbation` part de la moyenne honnête et la décale dans une direction $\boldsymbol{e}$. L'attaque choisit le plus grand décalage qui reste sélectionné par l'agrégateur ciblé :

$$\boldsymbol{b} = B(\gamma_m) = \bar{\boldsymbol{g}} + \gamma_m \boldsymbol{e}, \qquad \gamma_m = \sup\{\gamma \geq 0 : B(\gamma) \text{ est sélectionné}\}.$$

Par défaut, $\boldsymbol{e}$ est le vecteur de base de la coordonnée présentant la plus forte variance honnête. Les f attaquants envoient ce même vecteur.

Modèle de menace et résilience

Parmi les n gradients reçus, au plus f peuvent être arbitraires. Ils peuvent être coordonnés et dépendre des gradients honnêtes. Ces derniers sont supposés indépendants, de même loi, et d'espérance $\boldsymbol{g}$.

Une règle F est (α, f)-résiliente si son espérance reste orientée vers le vrai gradient et si ses fluctuations restent contrôlées :

$$\langle \mathbb{E}[F], \boldsymbol{g} \rangle \geq (1 - \sin\alpha)\|\boldsymbol{g}\|^2 > 0, \qquad 0 \leq \alpha < \frac{\pi}{2}.$$

Les moments $\mathbb{E}\|F\|^r$, pour $r = 2, 3, 4$, doivent aussi être bornés en fonction des moments des gradients honnêtes.

Cette propriété ne suffit pas seule à prouver la convergence : le bruit honnête et le calendrier du pas d'apprentissage interviennent également.

Écarts entre publication et implémentation

Règle	Publication	Bibliothèque
`MultiKrum`	Sélection itérative, avec recalcul après chaque retrait.	Scores calculés une fois, puis moyenne des m plus petits.
`Bulyan`	Sélection de $n - 2f$ candidats.	Sélection de $n - 2f - 2$ candidats, puis filtrage coordonnée par coordonnée.

Pour $m = 1$, `MultiKrum` coïncide avec `Krum`. Les résultats des expériences caractérisent les variantes effectivement codées. Une garantie théorique ne se transfère pas automatiquement à une variante modifiée.

Ce que montrent les deux expériences

Spambase, paramètres de 2017

$n = 20$, $f = 6$, `MultiKrum` avec $m = 14$, `SignFlip` d'amplitude 10. La moyenne diverge dans cette simulation ; `MultiKrum` poursuit l'apprentissage. Le réglage $m = 14$ dépasse la borne théorique $m \leq 5$.

MNIST, attaque de 2018

$n = 39$, $f = 9$, $\gamma = 4$. L'exactitude plafonne vers 90 % pour `Krum`, contre environ 94 % pour `Bulyan` et 95 % pour la référence saine.

Ces courbes illustrent les phénomènes visés dans ces réglages. Elles n'établissent ni une garantie théorique ni une stabilité statistique sur plusieurs graines.

Fidélité, répétabilité et portée

- **Fidélité :** conserver les mécanismes utiles du protocole étudié et documenter les adaptations de la simulation.
- **Répétabilité :** une graine fixe les tirages pseudo-aléatoires. Dans l'environnement déterministe documenté, relancer la même configuration donne les mêmes résultats.
- **Limite :** une graine ne mesure pas la variabilité entre exécutions. Il faudrait plusieurs graines et une analyse statistique.
- **Périmètre :** les expériences tournent séquentiellement sur une machine ; les échanges entre workers sont simulés.